

Ultimate Exam-Style Problem Bank

20 Sets × 7 Questions — Very Easy → Extreme, University-Exam Style

Final 3-Day Sprint — DIU Take-OFF Contest Prep

Language: **C** | 140 original problems, difficulty ramps 1 → 7 within every set

Every set has 7 problems: Q1 is trivial (basic I/O), Q2-Q3 are easy-medium, Q4 is medium, Q5 is hard, and Q6-Q7 push further — genuinely tough, often multi-concept problems written in full university-exam style (context, Input, Output, Constraints, Sample), well beyond the real contest's difficulty so the actual exam feels easier by comparison. Sets are ordered so early sets build fundamentals and later sets (15-20) combine everything into contest-strength and beyond-contest-strength problems, including full rebuilds and fusions of the hardest problems from your real contest (the H-style and J-style problems).

Table of Contents

Set 1	I/O & Arithmetic Basics
Set 2	Conditionals & Strings Basics
Set 3	Arrays: Frequency, Min/Max, Search
Set 4	Bitwise Operations & Number Properties
Set 5	Sorting Fundamentals
Set 6	Basic Geometry
Set 7	Prefix Sums & Range Queries
Set 8	Greedy Algorithms — Introduction
Set 9	Binary Search — On Arrays and On the Answer
Set 10	Number Theory: GCD, LCM, Primes, Divisors
Set 11	Strings — Intermediate
Set 12	Graphs — BFS/DFS Introduction
Set 13	Dynamic Programming — Introduction
Set 14	Combinatorics & Modular Math
Set 15	Advanced Greedy + Prefix Sums (World T20 X Team style)
Set 16	Advanced Graphs — Implicit Graphs & Multi-source
Set 17	Advanced Number Theory
Set 18	Advanced Dynamic Programming
Set 19	Mixed Practice Contest — Round A
Set 20	Final Mock Contest — Round B (Cumulative, hardest set)

Set 1

I/O & Arithmetic Basics

Set 1 - Q1: Add Two Numbers

VERY EASY · Q1

Fahim has just joined the CS department and is writing his very first C programs for orientation week. His mentor starts him off with the basics: read two numbers scribbled on his notebook and report their total. Read two integers A and B on one line and print their sum.

Input	Two space-separated integers A, B.
Output	A single integer: A + B.
Constraints	$-10^9 \leq A, B \leq 10^9$
Sample Input	3 5
Sample Output	8

Set 1 - Q2: Even or Odd

EASY · Q2

Next, the mentor hands Fahim a stack of roll-number cards and asks him to write a quick program that sorts them into odd and even piles. Read an integer N and print "EVEN" if it is even, otherwise print "ODD".

Input	One integer N.
Output	"EVEN" or "ODD".
Constraints	$-10^9 \leq N \leq 10^9$
Sample Input	7
Sample Output	ODD

Set 1 - Q3: Simple Interest

MEDIUM · Q3

The department office is calculating workshop stipends. Given a student's principal amount, an interest rate, and a duration, Fahim must compute the simple interest so the office can print stipend cheques. Given principal P, rate R (percent per year) and time T (years), compute simple interest = $P \cdot R \cdot T / 100$, printed rounded down to the nearest integer.

Input	Three integers P R T on one line.
Output	One integer: floor(simple interest).
Constraints	$1 \leq P \leq 10^6, 1 \leq R \leq 100, 1 \leq T \leq 50$
Sample Input	1000 5 2
Sample Output	100

Set 1 - Q4: Largest of Three

MEDIUM-HARD · Q4

Three friends compare how much they spent on lunch this week. Fahim's program should tell them who spent the most, so the others know who's buying next time. Read three integers and print the largest one.

Input	Three integers A B C.
Output	The maximum of the three.
Constraints	$-10^9 \leq A, B, C \leq 10^9$
Sample Input	4 9 2
Sample Output	9

Set 1 - Q5: Digit Sum Loop**HARD · Q5**

As an icebreaker, the mentor asks everyone to reduce their student ID number to a single "lucky digit" by repeatedly summing its digits — Fahim needs a program to automate this for the whole batch. Read a positive integer N and repeatedly replace N with the sum of its digits until N becomes a single digit (0-9). Print the final single digit. (This is the "digital root" — try to also think of the $O(1)$ math shortcut using mod 9, not just simulation.)

Input	One integer N .
Output	A single digit, the digital root of N .
Constraints	$1 \leq N \leq 10^{18}$
Sample Input	9875
Sample Output	2 (9+8+7+5=29 → 2+9=11 → 1+1=2)

Set 1 - Q6: Big Integer Addition (arbitrary precision)**VERY HARD · Q6**

For the closing challenge, two students exchange enormous made-up ID numbers, far too big for any built-in integer type, and want to know their sum without a calculator. Read two non-negative integers A and B given as strings (they may have up to 200 digits — far beyond what fits in any built-in integer type). Print $A+B$ as a string, computed via manual digit-by-digit addition with carry, without using any big-integer library.

Input	Two strings A and B (digits only) on two lines.
Output	A string: the sum, no leading zeros (unless the sum is 0).
Constraints	$1 \leq A , B \leq 200$
Sample Input	99999999999999999999 1
Sample Output	100000000000000000000

Set 1 - Q7: Reconstruct N from K Statistics (constructive/exam-style)**EXTREME · Q7**

As a final riddle, the mentor gives Fahim only three clues about a hidden set of N numbers — their sum, their product, and their maximum — and challenges him to reconstruct any set of numbers that fits. You are told: the sum of N positive integers is S , their product is Pr , and the maximum is M . Determine whether such a set of N positive integers can exist, and if so, output one valid example (any valid array). This is a constructive reasoning problem — think about the tightest possible bounds (e.g. what's the minimum possible sum given $N-1$ values of 1 and one value M ?) before coding a search.

Input	Three integers N S Pr M on one line.
Output	"NO" if impossible, otherwise "YES" followed by N integers forming a valid example.
Constraints	$1 \leq N \leq 20, 1 \leq S, Pr, M \leq 10^9$
Sample Input	3 6 6 3
Sample Output	YES 1 2 3

Set 2

Conditionals & Strings Basics

Set 2 - Q1: Vowel or Consonant

VERY EASY · Q1

Rina is building a small spellchecking kiosk for the campus library. Her first test: given a single letter typed by a visitor, tell them whether it's a vowel or a consonant. Read a single lowercase English letter and print "VOWEL" or "CONSONANT".

Input	One lowercase letter.
Output	"VOWEL" or "CONSONANT".
Constraints	The character is a lowercase English letter.
Sample Input	e
Sample Output	VOWEL

Set 2 - Q2: String Length

EASY · Q2

A visitor wants to know how many characters they typed into the kiosk's search box before hitting enter — Rina's kiosk should report the length instantly. Read a line of text (no spaces) and print its length.

Input	One string S of lowercase letters.
Output	An integer: length of S.
Constraints	$1 \leq S \leq 1000$
Sample Input	hello
Sample Output	5

Set 2 - Q3: Reverse a String

MEDIUM · Q3

For a fun "mirror mode" feature, the kiosk should take whatever a visitor types and show it to them reversed, letter by letter. Read a string and print it reversed.

Input	One string S.
Output	S reversed.
Constraints	$1 \leq S \leq 10^5$
Sample Input	abcd
Sample Output	dcba

Set 2 - Q4: Grading System

MEDIUM-HARD · Q4

At the end of the semester, the library wants Rina's kiosk to convert numeric exam scores into letter grades for the results board. Given a score 0-100, print the grade: "A" for ≥ 80 , "B" for ≥ 60 , "C" for ≥ 40 , "F" otherwise.

Input	One integer score.
Output	One of A, B, C, F.
Constraints	$0 \leq \text{score} \leq 100$
Sample Input	72
Sample Output	B

Set 2 - Q5: Balanced Brackets (single type)**HARD · Q5**

A group of CS freshmen are experimenting with code snippets in the kiosk's text editor, and Rina needs the editor to flag whether their parentheses are properly balanced before they try to compile anything. Read a string made only of characters '(' and ')'. Print "YES" if the brackets are balanced (every open has a matching close, and never more closes than opens at any prefix), otherwise "NO".

Input	One string S of '(' and ')'.
Output	"YES" or "NO".
Constraints	$1 \leq S \leq 10^5$
Sample Input	<pre>((()))</pre>
Sample Output	YES

Set 2 - Q6: Multi-Bracket Validity with Minimum Fix Cost**VERY HARD · Q6**

The freshmen's code now uses three different kinds of brackets, and half of it doesn't balance at all — Rina's editor must now report the minimum number of characters needed to fix it, not just yes/no. Read a string of '(', ')', '[', ']', '{', '}'. If it's already balanced print "VALID". Otherwise print the MINIMUM number of characters you must insert to make it balanced (insertions only, no deletions or substitutions allowed).

Input	One string S of the six bracket characters.
Output	"VALID" or an integer: minimum insertions needed.
Constraints	$1 \leq S \leq 10^5$
Sample Input	<pre>([)]</pre>
Sample Output	2 (e.g. insert to get ([[]]) with matching, or similar – verify by hand)

Set 2 - Q7: Word Ladder Length (string transform, exam style)**EXTREME · Q7**

Impressed, the freshmen challenge Rina with a classic word puzzle from their algorithms class: transform one word into another one letter at a time, using only real dictionary words at every step, in as few steps as possible. Given a start word, an end word, and a dictionary of N words (all same length), find the length of the shortest transformation sequence from start to end, changing exactly one letter at a time, where every intermediate word must exist in the dictionary. Print -1 if impossible.

Input	Line1: start end. Line2: N. Next N lines: dictionary words.
Output	Length of shortest transformation sequence (number of words including start and end), or -1.
Constraints	$1 \leq N \leq 5000$, all words equal length ≤ 10
Sample Input	<pre>hit cog 6 hot dot dog lot log cog</pre>
Sample Output	5

Set 3

Arrays: Frequency, Min/Max, Search

Set 3 - Q1: Array Sum

VERY EASY · Q1

Tanvir is helping the department count votes for the best-project award. His first job: given every judge's marks, add them all up. Read N integers and print their sum.

Input	First line N , second line N integers.
Output	Sum of the array.
Constraints	$1 \leq N \leq 10^5$, $ a_i \leq 10^4$
Sample Input	4 1 2 3 4
Sample Output	10

Set 3 - Q2: Find Maximum

EASY · Q2

Among all the marks submitted, Tanvir needs to quickly find the single highest score awarded. Read N integers and print the maximum value.

Input	First line N , second line N integers.
Output	The maximum value.
Constraints	$1 \leq N \leq 10^5$
Sample Input	5 3 7 2 9 4
Sample Output	9

Set 3 - Q3: Count Occurrences

MEDIUM · Q3

One judge is suspected of favoring a particular project — Tanvir must count exactly how many times that project's score appears in the marks list. Read N integers and a query value X . Print how many times X appears in the array.

Input	First line N , second line N integers, third line X .
Output	One integer: count of X .
Constraints	$1 \leq N \leq 10^5$
Sample Input	5 1 2 2 3 2 2
Sample Output	3

Set 3 - Q4: Second Largest (distinct)**MEDIUM-HARD · Q4**

To choose the runner-up, Tanvir needs the second-highest DISTINCT score, since several projects tied for the top mark. Read N integers and print the second largest DISTINCT value. It is guaranteed at least 2 distinct values exist.

Input	First line N , second line N integers.
Output	The second largest distinct value.
Constraints	$2 \leq N \leq 10^5$
Sample Input	5 4 4 7 7 2
Sample Output	4

Set 3 - Q5: Array Has a Pair Summing to K**HARD · Q5**

A rumor says two projects' combined marks equal exactly the passing threshold K — Tanvir must check if any such pair exists among all submitted marks. Read N integers and integer K . Print "YES" if any two DIFFERENT positions $i \neq j$ have $a[i] + a[j] = K$, else "NO". Aim for $O(N)$ or $O(N \log N)$, not $O(N^2)$, since N can be up to 10^5 .

Input	First line N K , second line N integers.
Output	"YES" or "NO".
Constraints	$1 \leq N \leq 10^5$, $ a_i , K \leq 10^9$
Sample Input	5 9 2 7 4 1 5
Sample Output	YES

Set 3 - Q6: Count Triplets Summing to Zero**VERY HARD · Q6**

For a deeper audit, the committee wants to know how many GROUPS of three marks sum to exactly zero after marks were normalized around the class average (so some values are negative). Read N integers. Count the number of index triplets $(i < j < k)$ such that $a[i] + a[j] + a[k] = 0$. Aim for $O(N^2)$ using sorting + two pointers, not $O(N^3)$.

Input	Line1: N . Line2: N integers.
Output	Count of triplets summing to zero.
Constraints	$1 \leq N \leq 3000$, $ a_i \leq 10^4$
Sample Input	6 -1 0 1 2 -1 -4
Sample Output	2

Set 3 - Q7: Longest Subarray with Equal 0s and 1s After K Flips**EXTREME · Q7**

Finally, Tanvir is auditing a binary eligibility list (1 = eligible, 0 = not) and is allowed to "appeal" at most K non-eligible marks into eligible ones — he needs the longest stretch of eligible entries achievable. Read a binary array (0s and 1s) of size N and an integer K. You may flip at most K zeros to ones. Print the length of the longest contiguous subarray consisting only of 1s achievable after the flips. Use a sliding window, $O(N)$.

Input	Line1: N K. Line2: N binary values.
Output	Length of the longest all-1s subarray achievable.
Constraints	$1 \leq N \leq 10^5$, $0 \leq K \leq N$
Sample Input	11 2 1 1 1 0 0 0 1 1 1 1 0
Sample Output	5

Set 4

Bitwise Operations & Number Properties

Set 4 - Q1: Is Even (bitwise)

VERY EASY · Q1

Nusrat is debugging low-level embedded firmware for a robotics club project, where every operation must be a bitwise trick, not a shortcut library call. First up: check if a sensor reading is even, using only bitwise AND. Read N and print "EVEN" or "ODD" using only the bitwise AND operator ($N \& 1$), not the modulo operator.

Input	One integer N .
Output	"EVEN" or "ODD".
Constraints	$0 \leq N \leq 10^9$
Sample Input	10
Sample Output	EVEN

Set 4 - Q2: Count Set Bits

EASY · Q2

The firmware also needs a quick population count — how many bits are flipped on in a given sensor register. Read N and print how many bits are set to 1 in its binary representation.

Input	One integer N .
Output	Count of set bits.
Constraints	$0 \leq N \leq 10^9$
Sample Input	13
Sample Output	3 (1101)

Set 4 - Q3: Swap Without Temp

MEDIUM · Q3

A faulty wire means Nusrat can't use a temporary register when swapping two sensor values — she must swap them using pure XOR tricks. Read two integers A and B , swap their values using XOR (no temporary variable), and print them in swapped order.

Input	Two integers A B .
Output	B and A (swapped), space-separated.
Constraints	$0 \leq A, B \leq 10^9$
Sample Input	5 9
Sample Output	9 5

Set 4 - Q4: Power of Two Check**MEDIUM-HARD · Q4**

The robot's memory allocator only works efficiently on power-of-two block sizes, so Nusrat needs an instant check for whether a requested size is a power of two. Read N . Print "YES" if N is a power of two (1,2,4,8,...), else "NO". Solve using the bit trick $N \& (N-1) == 0$, in $O(1)$.

Input	One integer N .
Output	"YES" or "NO".
Constraints	$1 \leq N \leq 10^{18}$
Sample Input	64
Sample Output	YES

Set 4 - Q5: Single Non-Duplicate**HARD · Q5**

During a stress test, every sensor ID logs twice except ONE faulty sensor that logs only once — Nusrat must find it using $O(1)$ extra memory, no arrays allowed. Read N integers where every value appears EXACTLY TWICE except one value which appears exactly once. Find that value in $O(N)$ time and $O(1)$ extra space using XOR (do not use a hashmap/array counting approach).

Input	First line N (odd), second line N integers.
Output	The single non-duplicate value.
Constraints	$1 \leq N \leq 2 \times 10^5$, N is odd
Sample Input	5 4 1 2 1 2
Sample Output	4

Set 4 - Q6: Two Non-Duplicates (every other value appears twice)**VERY HARD · Q6**

The stress test gets worse: now TWO faulty sensors each log only once, while everything else logs twice — Nusrat must recover both faulty IDs, still using pure bitwise tricks. Read N integers where every value appears EXACTLY TWICE except TWO values which each appear exactly once. Find both unique values (in any order) in $O(N)$ time, $O(1)$ extra space, using bitwise tricks only.

Input	Line1: N (N is even+2 effectively). Line2: N integers.
Output	The two unique values, space-separated (either order).
Constraints	$1 \leq N \leq 2 \times 10^5$
Sample Input	6 1 2 1 3 2 5
Sample Output	3 5

Set 4 - Q7: Maximum XOR Subarray**EXTREME · Q7**

For the final calibration, Nusrat must scan a whole log of sensor readings and find the contiguous stretch whose XOR is the largest possible, to detect the noisiest window. Read N integers. Find the maximum XOR value of any contiguous subarray (XOR of all elements in that subarray). Use a prefix-XOR array plus a binary trie for $O(N \log(\maxVal))$, not brute force $O(N^2)$.

Input	Line1: N . Line2: N integers.
Output	Maximum XOR of any contiguous subarray.
Constraints	$1 \leq N \leq 10^5$, $0 \leq a_i \leq 10^9$
Sample Input	4 1 2 3 4
Sample Output	7

Set 5

Sorting Fundamentals

Set 5 - Q1: Sort Ascending

VERY EASY · Q1

Coach Imran is arranging players by height for the sports day parade. His first request: sort the whole line of player heights from shortest to tallest. Read N integers and print them sorted in non-decreasing order.

Input	First line N , second line N integers.
Output	N integers sorted ascending.
Constraints	$1 \leq N \leq 10^5$
Sample Input	4 3 1 4 2
Sample Output	1 2 3 4

Set 5 - Q2: Kth Smallest

EASY · Q2

Imran also wants to know: if the players are lined up shortest to tallest, who stands at exactly the K -th position? Read N integers and integer K . Print the K -th smallest value (1-indexed).

Input	First line N K , second line N integers.
Output	The K -th smallest value.
Constraints	$1 \leq K \leq N \leq 10^5$
Sample Input	5 3 7 2 9 4 1
Sample Output	4

Set 5 - Q3: Sort by Custom Rule

MEDIUM · Q3

For the awards ceremony, Imran needs students sorted by their event score (highest first), and any exact ties broken alphabetically by name. Read N students, each with a name (no spaces) and a score. Print names sorted by score descending; if scores tie, sort those names alphabetically ascending.

Input	First line N , then N lines of "name score".
Output	N names, one per line, in sorted order.
Constraints	$1 \leq N \leq 1000$, $0 \leq \text{score} \leq 100$
Sample Input	3 Amit 80 Rafi 90 Sadia 80
Sample Output	Rafi Amit Sadia

Set 5 - Q4: Minimum Swaps to Sort**MEDIUM-HARD · Q4**

The players are currently in a random, disorganized line, and Imran wants the MINIMUM number of swaps needed to sort them by height with the least reshuffling possible. Read N distinct integers (a permutation of $1..N$ is NOT guaranteed, just distinct values). Find the minimum number of swaps needed to sort the array ascending.

Input	First line N , second line N distinct integers.
Output	Minimum number of swaps.
Constraints	$1 \leq N \leq 10^5$
Sample Input	4 4 3 2 1
Sample Output	2

Set 5 - Q5: Merge Two Sorted Arrays In-Place-Style (K-way idea)**HARD · Q5**

Two separate class sections already have their own sorted height lists, and Imran needs them combined into one single sorted line for the whole parade. Read two already-sorted arrays A (size N) and B (size M). Print the merged sorted array of size $N+M$ without using a full $O((N+M) \log(N+M))$ sort — use the classic $O(N+M)$ merge technique from merge sort.

Input	Line1: N M . Line2: N integers (sorted). Line3: M integers (sorted).
Output	$N+M$ integers, merged and sorted.
Constraints	$1 \leq N, M \leq 10^5$
Sample Input	3 2 1 4 7 2 5
Sample Output	1 2 4 5 7

Set 5 - Q6: Count Inversions**VERY HARD · Q6**

Imran suspects the line has never been properly sorted — he wants to know exactly how many pairs of players are "out of order" relative to a fully sorted arrangement. Read N integers. Count the number of pairs $(i < j)$ such that $a[i] > a[j]$ (inversions). Use merge-sort-based counting for $O(N \log N)$, not $O(N^2)$.

Input	Line1: N . Line2: N integers.
Output	Total number of inversions.
Constraints	$1 \leq N \leq 10^5$
Sample Input	5 2 4 1 3 5
Sample Output	3

Set 5 - Q7: K-Way Merge of Sorted Lists (min-heap based)**EXTREME - Q7**

Now Imran has K different class sections, each already internally sorted by height, and he needs all of them merged into one grand sorted line for the final march-past. Read K sorted arrays (given consecutively). Merge them all into a single sorted array. Use a min-heap of size K (comparing current fronts) for $O(\text{total_elements} * \log K)$, not concatenating and sorting everything from scratch.

Input	Line1: K. Then for each of the K lists: a line with its size, then a line with its sorted elements.
Output	All elements merged into one fully sorted sequence, space-separated.
Constraints	$1 \leq K \leq 1000$, total elements $\leq 10^5$
Sample Input	2 3 1 4 5 3 2 3 6
Sample Output	1 2 3 4 5 6

Set 6

Basic Geometry

Set 6 - Q1: Distance Between Two Points

VERY EASY · Q1

Architect Priya is drafting blueprints for the new campus building using only integer coordinates. Her first task: measure the straight-line distance between two marked points on the blueprint. Read coordinates of two points (x_1, y_1) and (x_2, y_2) . Print the Euclidean distance with 2 decimal places.

Input	Four integers x_1 y_1 x_2 y_2 .
Output	Distance, printed with 2 decimals.
Constraints	$-10^4 \leq \text{coords} \leq 10^4$
Sample Input	0 0 3 4
Sample Output	5.00

Set 6 - Q2: Point Inside Circle

EASY · Q2

Priya is placing a circular fountain and needs to check whether a proposed bench location falls inside or on the edge of the fountain's boundary. Given a circle centered at (cx, cy) with radius r , and a point (px, py) , print "INSIDE" if the point is inside or on the circle, else "OUTSIDE".

Input	Five integers cx cy r px py .
Output	"INSIDE" or "OUTSIDE".
Constraints	$-10^4 \leq \text{values} \leq 10^4$, $r \geq 0$
Sample Input	0 0 5 3 4
Sample Output	INSIDE

Set 6 - Q3: Rectangle Area & Perimeter

MEDIUM · Q3

For the garden plot next to the building, Priya needs to quickly compute both the area and the perimeter of a rectangular flowerbed. Given an axis-aligned rectangle by its bottom-left (x_1, y_1) and top-right (x_2, y_2) corners, print its area and perimeter, space-separated.

Input	Four integers x_1 y_1 x_2 y_2 ($x_1 < x_2$, $y_1 < y_2$).
Output	Area and perimeter, space-separated.
Constraints	$0 \leq \text{coords} \leq 10^4$
Sample Input	0 0 4 3
Sample Output	12 14

Set 6 - Q4: Point Inside Axis-Aligned Rectangle (Multi-query)**MEDIUM-HARD · Q4**

Later, visitors will scan a QR code at various spots on the blueprint, and Priya's system must instantly tell whether each scanned point falls inside a given rectangular "garden zone" — for many scans at once. Given an axis-aligned rectangle by two OPPOSITE corners (which may be given in any order, not guaranteed bottom-left/top-right), answer Q queries: for each point, print "YES" if it lies inside or on the rectangle boundary, else "NO".

Input	Line1: x1 y1 x2 y2 (opposite corners). Line2: Q. Then Q lines of px py.
Output	Q lines, each "YES" or "NO".
Constraints	$1 \leq Q \leq 10^5$, $-10^9 \leq \text{coords} \leq 10^9$
Sample Input	5 5 1 1 2 3 3 10 10
Sample Output	YES NO

Set 6 - Q5: Triangle Classification**HARD · Q5**

While reviewing a triangular courtyard design, Priya wants to know both its shape category (equilateral, isosceles, or scalene) and whether it happens to be a right triangle. Given the lengths of three sides (they always form a valid triangle), print "EQUILATERAL", "ISOSCELES", or "SCALENE". Additionally, on a second line print "RIGHT" if it's also a right triangle (within the given integer side lengths, exact check using $a^2+b^2=c^2$), otherwise print "NOT-RIGHT".

Input	Three integers a b c.
Output	Two lines as described.
Constraints	$1 \leq a, b, c \leq 10^4$
Sample Input	3 4 5
Sample Output	SCALENE RIGHT

Set 6 - Q6: Convex Polygon Area (Shoelace Formula)**VERY HARD · Q6**

For an irregular courtyard shaped like a simple polygon, Priya needs its exact area calculated directly from its corner coordinates. Given N vertices of a simple polygon in order (either clockwise or counter-clockwise), print its area using the Shoelace formula, with 2 decimal places.

Input	Line1: N. Next N lines: x y (vertex coordinates in order).
Output	The polygon area, 2 decimals.
Constraints	$3 \leq N \leq 1000$, $-10^4 \leq \text{coords} \leq 10^4$
Sample Input	4 0 0 4 0 4 4 0 4
Sample Output	16.00

Set 6 - Q7: Closest Pair of Points (Divide and Conquer)**EXTREME - Q7**

Finally, safety regulations require the two CLOSEST security cameras (out of hundreds placed as points on the blueprint) to be identified, so overlapping coverage can be reduced. Given N points in the plane, find the minimum Euclidean distance between any two distinct points. N can be up to 10^5 — an $O(N^2)$ brute force is too slow; use the classic divide-and-conquer closest-pair algorithm ($O(N \log N)$), or a sweep-line with a sorted auxiliary structure.

Input	Line1: N . Next N lines: $x\ y$.
Output	Minimum distance, 4 decimal places.
Constraints	$2 \leq N \leq 10^5$, $-10^9 \leq \text{coords} \leq 10^9$
Sample Input	4 0 0 3 4 1 1 100 100
Sample Output	1.4142

Set 7

Prefix Sums & Range Queries

Set 7 - Q1: Range Sum Query (single)

VERY EASY · Q1

Data analyst Shuvo is auditing a company's daily sales figures. First request: total sales between day L and day R. Read N integers, then a range [L,R] (1-indexed, inclusive). Print the sum of a[L..R].

Input	Line1: N. Line2: N integers. Line3: L R.
Output	Sum of the range.
Constraints	$1 \leq L \leq R \leq N \leq 10^5$
Sample Input	5 1 2 3 4 5 2 4
Sample Output	9

Set 7 - Q2: Multiple Range Sum Queries

EASY · Q2

Management now wants MANY such range totals answered quickly, for dozens of different day ranges, without recomputing from scratch each time. Read N integers and Q queries, each a range [L,R]. For each query print the sum of a[L..R]. Build a prefix sum array first so each query is $O(1)$ — don't recompute sums from scratch each time.

Input	Line1: N Q. Line2: N integers. Next Q lines: L R.
Output	Q integers, one sum per line.
Constraints	$1 \leq N, Q \leq 10^5$
Sample Input	5 2 1 2 3 4 5 1 3 2 5
Sample Output	6 14

Set 7 - Q3: Equilibrium Index

MEDIUM · Q3

Shuvo also wants to find a day in the sales record where everything sold BEFORE that day exactly balances everything sold AFTER it — a natural "equilibrium point" for the fiscal report. Find any index i (0-indexed) such that the sum of elements before i equals the sum of elements after i. Print -1 if none exists.

Input	Line1: N. Line2: N integers.
Output	An equilibrium index, or -1.
Constraints	$1 \leq N \leq 10^5$
Sample Input	5 -7 1 5 2 -4
Sample Output	3

Set 7 - Q4: Difference Array Range Updates**MEDIUM-HARD · Q4**

A promotional campaign will add a fixed bonus to sales across various date ranges, and Shuvo needs the final adjusted sales figures after all these range-wide bonuses are applied. Start with an array of N zeros. Process Q update operations, each adding value V to every element in range $[L,R]$ (1-indexed, inclusive). After all updates, print the final array. Use the difference-array technique so each update is $O(1)$, not $O(N)$.

Input	Line1: N Q . Next Q lines: L R V .
Output	The final array, N space-separated integers.
Constraints	$1 \leq N, Q \leq 10^5$, $ V \leq 10^4$
Sample Input	5 2 1 3 2 2 5 3
Sample Output	2 5 5 3 3

Set 7 - Q5: Maximum Subarray Sum (Kadane's)**HARD · Q5**

Looking for the best possible sales streak (even accounting for bad, negative-adjustment days), Shuvo wants the maximum total of any single contiguous stretch of days. Read N integers (can be negative) and print the maximum possible sum of a contiguous, non-empty subarray.

Input	Line1: N . Line2: N integers.
Output	The maximum subarray sum.
Constraints	$1 \leq N \leq 10^5$, $ a_i \leq 10^4$
Sample Input	8 -2 1 -3 4 -1 2 1 -5
Sample Output	6 (subarray 4, -1, 2, 1)

Set 7 - Q6: 2D Prefix Sum (Submatrix Sum Queries)**VERY HARD · Q6**

The sales data is now a full grid — rows are branches, columns are days — and Shuvo needs fast answers to "what were total sales in branches $R1$ - $R2$ during days $C1$ - $C2$?" for many such rectangular queries. Given an $N \times M$ matrix and Q queries, each defining a rectangular submatrix by its top-left and bottom-right corners, print the sum of all elements in that submatrix. Precompute a 2D prefix sum so each query is $O(1)$.

Input	Line1: N M . Next N lines: M integers each. Line: Q . Next Q lines: $r1$ $c1$ $r2$ $c2$ (1-indexed).
Output	Q lines: sum of the submatrix for each query.
Constraints	$1 \leq N, M \leq 1000$, $1 \leq Q \leq 10^5$
Sample Input	3 3 1 2 3 4 5 6 7 8 9 1 1 1 2 2
Sample Output	12

Set 7 - Q7: Range Sum With Point Updates (Fenwick/BIT)**EXTREME · Q7**

Finally, some sales figures get CORRECTED after the fact (a data-entry fix), interleaved with new range-sum requests — Shuvo needs a structure that handles both live updates and fast range queries. Read N integers, then process Q operations of two types: "U i v " (set $a[i] = v$) or "Q l r " (print sum of $a[l..r]$). Since updates are interleaved with queries, a plain prefix-sum array won't work efficiently — implement a Fenwick Tree (Binary Indexed Tree) for $O(\log N)$ per operation.

Input	Line1: N Q . Line2: N integers. Next Q lines: either "U i v " or "Q l r ".
Output	One line per query operation: the requested sum.
Constraints	$1 \leq N, Q \leq 10^5$
Sample Input	5 3 1 2 3 4 5 Q 1 5 U 3 10 Q 1 5
Sample Output	15 22

Set 8

Greedy Algorithms — Introduction

Set 8 - Q1: Maximum Coins (largest first)

VERY EASY · Q1

Cashier Mitu at the campus canteen only stocks coins of a few fixed denominations, in unlimited supply, and wants to give change using the FEWEST coins possible for any amount. You have coins of denominations 1, 5, 10, 25 (unlimited supply of each). Given an amount N , print the MINIMUM number of coins needed to make exactly N .

Input	One integer N .
Output	Minimum coin count.
Constraints	$1 \leq N \leq 10^6$
Sample Input	30
Sample Output	2 (25+5)

Set 8 - Q2: Activity Selection

EASY · Q2

The seminar hall is booked back-to-back all day, and Mitu (moonlighting as room coordinator) must pick the MAXIMUM number of non-overlapping sessions that can be scheduled in one hall. Given N activities each with a start and end time, print the MAXIMUM number of non-overlapping activities you can attend (an activity ending at time t and another starting at time t may both be attended).

Input	Line1: N . Next N lines: start end.
Output	Maximum number of activities.
Constraints	$1 \leq N \leq 10^5$
Sample Input	4 1 3 2 4 3 5 0 6
Sample Output	2

Set 8 - Q3: Minimum Platforms

MEDIUM · Q3

With multiple overlapping train arrivals expected during a university trip, Mitu needs to know the MINIMUM number of platforms required so no train has to wait. Given arrival and departure times of N trains at a station, find the MINIMUM number of platforms required so that no train waits.

Input	Line1: N . Line2: N arrival times. Line3: N departure times.
Output	Minimum platforms needed.
Constraints	$1 \leq N \leq 10^5$
Sample Input	3 900 940 950 910 1200 1120
Sample Output	2

Set 8 - Q4: Fractional Knapsack**MEDIUM-HARD · Q4**

Mitu is packing a fragile gift box (the "knapsack") for a farewell event and, since items can be split into fractions (like ribbon lengths or fabric), wants to maximize total sentimental value within a weight limit. Given N items each with a weight and value, and a knapsack capacity W , you may take fractions of items. Maximize total value. Print the answer rounded to 2 decimal places.

Input	Line1: N W . Next N lines: weight value.
Output	Maximum value, 2 decimals.
Constraints	$1 \leq N \leq 10^5$, $1 \leq W \leq 10^6$
Sample Input	3 50 10 60 20 100 30 120
Sample Output	240.00

Set 8 - Q5: Job Sequencing with Deadlines**HARD · Q5**

For an exam-day printing queue, each job has a deadline and takes exactly one time slot on the single printer — Mitu wants to schedule jobs to MAXIMIZE total priority points earned before deadlines are missed. Given N jobs, each with a deadline and a profit, and each job takes exactly 1 unit of time on a single machine, schedule jobs (each job must finish by its deadline or be skipped) to MAXIMIZE total profit. Print the maximum achievable profit.

Input	Line1: N . Next N lines: deadline profit.
Output	Maximum total profit.
Constraints	$1 \leq N \leq 10^5$, $1 \leq \text{deadline} \leq N$
Sample Input	4 2 100 1 19 2 27 1 25
Sample Output	152 (jobs with profit 100 and 25, or 100 and 27 depending on slot order – think carefully about tie-breaking)

Set 8 - Q6: Huffman Encoding Total Length**VERY HARD · Q6**

For efficient exam-result SMS broadcasts, Mitu wants to build a Huffman-style encoding of student names based on how often each is transmitted, minimizing total broadcast length. Given N characters with their frequencies, build a Huffman tree (always merge the two smallest-frequency nodes) and print the total encoded length in bits (sum of frequency $\times$ depth for each character).

Input	Line1: N . Line2: N frequencies.
Output	Total encoded length in bits.
Constraints	$1 \leq N \leq 10^5$
Sample Input	4 5 9 12 13
Sample Output	76 (verify by building the tree)

Set 8 - Q7: Gas Station Circular Tour**EXTREME - Q7**

Finally, coordinating a relay road-trip with limited fuel at each rest stop, Mitu needs to find a valid starting stop from which the whole circular route can be completed without running out of gas. Given N gas stations arranged in a circle, each with $\text{gas}[i]$ available and $\text{cost}[i]$ to travel to the next station, determine the starting station index (0-indexed) from which you can complete the full circular tour, or print -1 if impossible. Solve in $O(N)$, not $O(N^2)$.

Input	Line1: N . Line2: N gas values. Line3: N cost values.
Output	Starting index (0-indexed), or -1.
Constraints	$1 \leq N \leq 10^5$
Sample Input	5 1 2 3 4 5 3 4 5 1 2
Sample Output	3

Set 9

Binary Search — On Arrays and On the Answer

Set 9 - Q1: Binary Search Basic

VERY EASY · Q1

Librarian Arif keeps book ID numbers sorted on the shelf. Given a visitor's requested ID, Arif wants to instantly report its shelf position, or say it isn't there. Read a sorted array of N integers and a query value X . Print the 1-indexed position of X if it exists, else print -1.

Input	Line1: N . Line2: N sorted integers. Line3: X .
Output	Position of X , or -1.
Constraints	$1 \leq N \leq 10^5$
Sample Input	5 2 4 6 8 10 8
Sample Output	4

Set 9 - Q2: First and Last Occurrence

EASY · Q2

Since some book IDs are duplicated across branch copies, Arif now needs both the FIRST and LAST shelf positions of a requested ID. Read a sorted array (may contain duplicates) of N integers and a query X . Print the first and last 1-indexed positions of X , or "-1 -1" if not present.

Input	Line1: N . Line2: N sorted integers. Line3: X .
Output	Two integers: first and last positions.
Constraints	$1 \leq N \leq 10^5$
Sample Input	6 1 2 2 2 3 5 2
Sample Output	2 4

Set 9 - Q3: Square Root (integer floor)

MEDIUM · Q3

Arif is estimating how many square display cases (of side length s) fit under a shelf of a given area N , and needs $\text{floor}(\sqrt{N})$ computed without any built-in square-root function. Read a non-negative integer N . Print $\text{floor}(\sqrt{N})$ without using any built-in sqrt function — use binary search on the answer.

Input	One integer N .
Output	$\text{floor}(\sqrt{N})$.
Constraints	$0 \leq N \leq 10^{18}$
Sample Input	10
Sample Output	3

Set 9 - Q4: Minimize the Maximum (Painter's Partition style)**MEDIUM-HARD · Q4**

K painters are assigned to paint a row of boards of different lengths, working in parallel, and Arif (also the event's part-time coordinator) wants to MINIMIZE the maximum work assigned to any single painter. Given N boards with lengths $a_1..a_N$ and K painters, each painter paints a contiguous set of boards, and all painters work in parallel. Minimize the maximum total length painted by any single painter. Binary search on the answer (the max length), with a greedy feasibility check.

Input	Line1: N K. Line2: N integers.
Output	The minimized maximum length.
Constraints	$1 \leq K \leq N \leq 10^5$
Sample Input	5 3 10 20 30 40 50
Sample Output	60

Set 9 - Q5: Kth Smallest Element in a Sorted Matrix (via binary search on value)**HARD · Q5**

Every row and column in the library's shelving grid is sorted, and Arif wants to find the K-th smallest book ID across the entire grid without sorting everything from scratch. Given an $N \times N$ matrix where every row and every column is sorted ascending, and an integer K, find the K-th smallest element in the whole matrix. Solve using binary search on the VALUE (not the index), with a counting function, in better than $O(N^2 \log(N^2))$ sorting time.

Input	Line1: N K. Next N lines: N integers each (row of the matrix).
Output	The K-th smallest value.
Constraints	$1 \leq N \leq 300, 1 \leq K \leq N^2$
Sample Input	3 5 1 5 9 10 11 13 12 13 15
Sample Output	11

Set 9 - Q6: Aggressive Cows (Maximize Minimum Distance)**VERY HARD · Q6**

To keep grazing cows at the campus farm from fighting, Arif (who also volunteers there) must place them in stalls to MAXIMIZE the minimum distance between any two cows. Given N stall positions and C cows, place all C cows into stalls to MAXIMIZE the minimum distance between any two cows. Binary search on the answer (the minimum distance), with a greedy feasibility check placing cows one by one.

Input	Line1: N C. Line2: N stall positions.
Output	The maximized minimum distance.
Constraints	$2 \leq C \leq N \leq 10^5$
Sample Input	5 3 1 2 4 8 9
Sample Output	3

Set 9 - Q7: Median of Two Sorted Arrays in $O(\log(\min(N,M)))$ **EXTREME - Q7**

Two branches each hand Arif a separately sorted list of book IDs, and he needs the MEDIAN of the combined collection, fast, without physically merging both lists. Given two sorted arrays of sizes N and M , find the median of the combined array. You MUST achieve $O(\log(\min(N,M)))$ time (binary searching the partition point) — a full merge is too slow for the intended constraints.

Input	Line1: N M . Line2: N sorted integers. Line3: M sorted integers.
Output	The median, printed with 1 decimal place if it's a .5 value, otherwise as an integer.
Constraints	$1 \leq N, M \leq 10^6$
Sample Input	2 3 1 3 2 4 5
Sample Output	3.0

Set 10

Number Theory: GCD, LCM, Primes, Divisors

Set 10 - Q1: GCD of Two Numbers

VERY EASY · Q1

Number theory TA Farhan is grading a stack of assignments. First question: find the GCD of two integers using the Euclidean algorithm. Read two positive integers A and B. Print their GCD using the Euclidean algorithm.

Input	Two integers A B.
Output	GCD(A,B).
Constraints	$1 \leq A, B \leq 10^{18}$
Sample Input	48 18
Sample Output	6

Set 10 - Q2: LCM of Two Numbers

EASY · Q2

Next question on the same assignment: find the LCM of two integers, being careful about overflow for large inputs. Read two positive integers A and B. Print their LCM. Be careful of overflow — use $LCM = A/GCD(A,B)*B$, and use long long / a wider type in C.

Input	Two integers A B.
Output	LCM(A,B).
Constraints	$1 \leq A, B \leq 10^6$
Sample Input	4 6
Sample Output	12

Set 10 - Q3: Prime Check (fast)

MEDIUM · Q3

A student's program needs to check primality of large numbers efficiently for a cryptography demo Farhan is preparing. Read an integer N. Print "PRIME" or "NOT PRIME". Your check must run in $O(\sqrt{N})$, not $O(N)$.

Input	One integer N.
Output	"PRIME" or "NOT PRIME".
Constraints	$2 \leq N \leq 10^{12}$
Sample Input	97
Sample Output	PRIME

Set 10 - Q4: Count Divisors of Every Number up to N (Sieve style)**MEDIUM-HARD · Q4**

For a number-theory poster, Farhan wants the divisor count of EVERY integer from 1 to N, computed efficiently with a sieve-style approach rather than one-by-one factorization. Read N. For every integer from 1 to N, compute the number of divisors it has, using a sieve-like $O(N \log N)$ approach (do NOT check each number individually in $O(\sqrt{i})$) — that's also acceptable but the sieve approach is the intended lesson). Print the divisor counts space-separated.

Input	One integer N.
Output	N integers: divisor counts for 1..N.
Constraints	$1 \leq N \leq 10^6$
Sample Input	6
Sample Output	1 2 2 3 2 4

Set 10 - Q5: GCD of an Increasing Subsequence of Length K (mini version)**HARD · Q5**

As a teaser for the final exam, Farhan gives a small array and asks: among all increasing subsequences of a given length K, what's the largest possible GCD? Read an array of N integers and an integer K. Among all increasing subsequences of length exactly K, print the MAXIMUM possible GCD of the subsequence's elements, or -1 if no increasing subsequence of length K exists. (Hint: for each candidate divisor d from large to small, keep only elements divisible by d, and check if an increasing subsequence of length $\geq K$ exists among them — this is a direct simplified version of the hardest problem from your real contest set.)

Input	Line1: N K. Line2: N integers.
Output	Maximum GCD, or -1.
Constraints	$1 \leq K \leq N \leq 500$, $1 \leq a_i \leq 500$
Sample Input	5 3 4 8 6 12 3
Sample Output	4 (e.g. 4, 8, 12 is increasing with GCD 4)

Set 10 - Q6: Chinese Remainder Theorem (2 congruences)**VERY HARD · Q6**

Farhan's cryptography demo also needs the Chinese Remainder Theorem: given two modular constraints on an unknown number, find the smallest value satisfying both. Given two congruences $X \equiv R1 \pmod{M1}$ and $X \equiv R2 \pmod{M2}$ where M1 and M2 are coprime, find the smallest non-negative X satisfying both, modulo $M1 \cdot M2$.

Input	Four integers R1 M1 R2 M2.
Output	The smallest non-negative X (mod $M1 \cdot M2$) satisfying both congruences.
Constraints	$0 \leq R1 < M1$, $0 \leq R2 < M2 \leq 10^6$, $\gcd(M1, M2) = 1$
Sample Input	2 3 3 5
Sample Output	8

Set 10 - Q7: Count Numbers $\leq N$ Coprime to N in a Range, Fast (Möbius-based)**EXTREME · Q7**

Finally, for a probability demo, Farhan wants to count how many integers in a (potentially huge) range are coprime to a given N , without scanning the whole range one by one. Given N and a range $[L,R]$, count how many integers in $[L,R]$ are coprime to N . N can be large, and the range $[L,R]$ can be huge (up to 10^{18}), so you must factorize N (up to $\sqrt{N}$) and then use inclusion-exclusion over N 's distinct prime factors (Möbius-style) rather than iterating the whole range.

Input	Three integers $N\ L\ R$.
Output	Count of integers in $[L,R]$ coprime to N .
Constraints	$1 \leq N \leq 10^{12}$, $1 \leq L \leq R \leq 10^{18}$
Sample Input	6 1 20
Sample Output	7 (numbers coprime to 6 in $[1,20]$: 1,5,7,11,13,17,19)

Set 11

Strings — Intermediate

Set 11 - Q1: Is Palindrome

VERY EASY · Q1

Text-processing intern Meherun is building a mini text-analysis tool. First feature: check if a given word reads the same forwards and backwards. Read a string. Print "YES" if it reads the same forwards and backwards, else "NO".

Input	One string S.
Output	"YES" or "NO".
Constraints	$1 \leq S \leq 10^5$
Sample Input	level
Sample Output	YES

Set 11 - Q2: Character Frequency

EASY · Q2

Next, Meherun's tool should report the single most frequent character typed into a document (breaking ties alphabetically). Read a string of lowercase letters. Print the most frequent character. If there's a tie, print the alphabetically smallest among the tied characters.

Input	One string S.
Output	A single character.
Constraints	$1 \leq S \leq 10^5$
Sample Input	banana
Sample Output	a

Set 11 - Q3: Anagram Check

MEDIUM · Q3

A plagiarism-check feature needs to verify whether one submitted title is just a rearranged (anagram) version of another. Read two strings A and B of equal length. Print "YES" if B is an anagram of A (same letters, same counts, any order), else "NO".

Input	Two strings A and B on two lines.
Output	"YES" or "NO".
Constraints	$1 \leq A = B \leq 10^5$
Sample Input	listen silent
Sample Output	YES

Set 11 - Q4: Longest Substring Without Repeating Characters

MEDIUM-HARD · Q4

For a "typing test" feature, Meherun's tool must find the longest stretch of typed text with no character repeated. Read a string S. Print the length of the longest substring that has no repeated characters. Aim for $O(N)$ using a sliding window, not $O(N^2)$.

Input	One string S of lowercase letters.
Output	An integer: the length.
Constraints	$1 \leq S \leq 10^5$
Sample Input	abcabcbb
Sample Output	3 ("abc")

Set 11 - Q5: Count Distinct Substrings of a String of a Single Repeated Character**HARD · Q5**

The tool is now applied to a stress-test string made of a single repeated letter, and must answer: what is the length of the x -th lexicographically smallest distinct piece of that text? Given a string consisting only of the character 'a', repeated N times, and Q queries each giving an integer x , print the length of the x -th lexicographically smallest DISTINCT substring for each query. (This mirrors StringOfAaaa directly — derive the closed-form using triangular numbers plus binary search, since N can be up to 10^9 .)

Input	Line1: N Q . Next Q lines: one integer x each.
Output	Q lines: length of the x -th smallest substring, or -1 if x exceeds total distinct substrings.
Constraints	$1 \leq N \leq 10^9$, $1 \leq Q \leq 10^5$, $1 \leq x \leq 10^{18}$
Sample Input	4 2 1 5
Sample Output	1 3

Set 11 - Q6: Z-Function / Pattern Occurrences**VERY HARD · Q6**

For fast search-and-highlight, Meherun's tool needs every occurrence of a search pattern inside a huge document, computed efficiently rather than scanning character by character. Given a text T and pattern P , find every starting index (0-indexed) in T where P occurs as a substring. Use the Z-function or KMP algorithm for $O(|T|+|P|)$, not naive $O(|T|*|P|)$.

Input	Two strings T and P on two lines.
Output	All starting indices where P occurs in T , space-separated (empty line if none).
Constraints	$1 \leq P \leq T \leq 10^6$
Sample Input	ababcabab ab
Sample Output	0 2 5 7

Set 11 - Q7: Longest Palindromic Substring in $O(N)$ (Manacher's Algorithm)**EXTREME · Q7**

Finally, for an "interesting text" detector, the tool must find the longest stretch of a document that reads the same forwards and backwards, out of potentially a million characters. Given a string S , print the length of its longest palindromic SUBSTRING (contiguous, not subsequence). N can be up to 10^6 , so an $O(N^2)$ expand-around-center approach is too slow — implement Manacher's algorithm for $O(N)$.

Input	One string S .
Output	Length of the longest palindromic substring.
Constraints	$1 \leq S \leq 10^6$
Sample Input	babad
Sample Output	3 ("bab" or "aba")

Set 12

Graphs — BFS/DFS Introduction

Set 12 - Q1: Count Connected Components

VERY EASY · Q1

Network admin Rakib is mapping the university's internal server network. First task: count how many separate isolated clusters of machines exist. Given an undirected graph with N nodes and M edges, print the number of connected components.

Input	Line1: N M . Next M lines: u v (an edge).
Output	Number of connected components.
Constraints	$1 \leq N \leq 10^5$, $0 \leq M \leq 2 \times 10^5$
Sample Input	5 2 1 2 2 3
Sample Output	3

Set 12 - Q2: Shortest Path in an Unweighted Graph

EASY · Q2

Rakib also needs the shortest number of hops from the main server to every other machine in the network. Given an undirected unweighted graph with N nodes and M edges, and a source node S , print the shortest distance (number of edges) from S to every node $1..N$. Print -1 for unreachable nodes.

Input	Line1: N M S . Next M lines: u v .
Output	N integers: distances from S to each node $1..N$.
Constraints	$1 \leq N \leq 10^5$, $0 \leq M \leq 2 \times 10^5$
Sample Input	4 3 1 1 2 2 3 3 4
Sample Output	0 1 2 3

Set 12 - Q3: Detect a Cycle in an Undirected Graph

MEDIUM · Q3

A junior admin suspects a faulty configuration created a routing loop — Rakib must check if the network contains any cycle at all. Given an undirected graph with N nodes and M edges, print "YES" if it contains a cycle, else "NO".

Input	Line1: N M . Next M lines: u v .
Output	"YES" or "NO".
Constraints	$1 \leq N \leq 10^5$, $0 \leq M \leq 2 \times 10^5$
Sample Input	4 4 1 2 2 3 3 4 4 1
Sample Output	YES

Set 12 - Q4: Bipartite Check**MEDIUM-HARD · Q4**

For a load-balancing experiment, Rakib wants to know if the network can be split into exactly two groups such that every connection goes between groups (never within one). Given an undirected graph with N nodes and M edges, print "YES" if it can be 2-colored (bipartite), else "NO". Use BFS/DFS coloring, and remember the graph might not be connected.

Input	Line1: N M . Next M lines: u v .
Output	"YES" or "NO".
Constraints	$1 \leq N \leq 10^5$, $0 \leq M \leq 2 \times 10^5$
Sample Input	4 4 1 2 2 3 3 4 4 1
Sample Output	YES

Set 12 - Q5: Reachability on a Jump Graph (SHA75-style)**HARD · Q5**

In the disaster-recovery wing, machines are numbered in a line, and from machine i you can only "failover" to $i+A[i]$ or $i-A[i]$ — Rakib needs to know if a backup machine is reachable from the primary one. You are given N positions in a line and an array A of size N . From position i (1-indexed) you may jump to position $i+A[i]$ or $i-A[i]$, as long as the target is between 1 and N . Given a query (start, end), print "YES" if end is reachable from start, else "NO". Answer Q independent queries. Use BFS/DFS per query, or precompute smartly if Q is large.

Input	Line1: N . Line2: N integers (array A). Line3: Q . Next Q lines: start end.
Output	Q lines: "YES" or "NO".
Constraints	$1 \leq N \leq 10^5$, $1 \leq Q \leq 100$
Sample Input	5 2 3 1 1 1 1 1 5
Sample Output	YES

Set 12 - Q6: Number of Islands (Connected Components in a Grid)**VERY HARD · Q6**

Zooming out to a satellite map of connected campus buildings (marked as a grid of land/water), Rakib needs to count how many separate "islands" of connected buildings exist. Given an $N \times M$ grid of '1' (land) and '0' (water), count the number of islands, where an island is a maximal group of '1's connected 4-directionally (not diagonally).

Input	Line1: N M . Next N lines: grid rows of '0'/'1'.
Output	Number of islands.
Constraints	$1 \leq N, M \leq 500$
Sample Input	4 5 11000 11000 00100 00011
Sample Output	3

Set 12 - Q7: Strongly Connected Components (Kosaraju's or Tarjan's)**EXTREME · Q7**

Finally, for the most critical subsystem, Rakib needs to identify tightly-coupled clusters of servers where every machine can reach every other machine in the same cluster (directed connections only). Given a directed graph with N nodes and M edges, print the number of strongly connected components (SCCs) — maximal sets of nodes where every node can reach every other node in the set.

Input	Line1: N M . Next M lines: u v (directed edge $u \rightarrow v$).
Output	Number of strongly connected components.
Constraints	$1 \leq N \leq 10^5$, $0 \leq M \leq 2 \times 10^5$
Sample Input	5 5 1 2 2 3 3 1 3 4 4 5
Sample Output	3

Set 13

Dynamic Programming — Introduction

Set 13 - Q1: Fibonacci (DP, not recursion)

VERY EASY · Q1

Puzzle-club president Dip is prepping brain-teasers for the club's game night. Warm-up: compute a large Fibonacci number efficiently, without letting recursion blow the stack. Read N . Print the N -th Fibonacci number ($F(1)=1$, $F(2)=1$) using an iterative DP array or two rolling variables — NOT plain recursion (too slow for large N).

Input	One integer N .
Output	$F(N)$.
Constraints	$1 \leq N \leq 90$
Sample Input	10
Sample Output	55

Set 13 - Q2: Climbing Stairs

EASY · Q2

Next puzzle: a player can climb a staircase 1 or 2 steps at a time — how many distinct ways are there to reach the top? You can climb 1 or 2 steps at a time. Given N total steps, print the number of distinct ways to reach the top.

Input	One integer N .
Output	Number of ways.
Constraints	$1 \leq N \leq 45$
Sample Input	4
Sample Output	5

Set 13 - Q3: 0/1 Knapsack

MEDIUM · Q3

For the "treasure bag" round, players choose items (each with a weight and value) to maximize total value without exceeding the bag's weight limit, and each item can only be picked once. Given N items with weights and values, and knapsack capacity W , find the maximum total value without exceeding W , where each item can be used at most once.

Input	Line1: N W . Next N lines: weight value.
Output	Maximum value.
Constraints	$1 \leq N \leq 500$, $1 \leq W \leq 10^4$
Sample Input	3 50 10 60 20 100 30 120
Sample Output	220

Set 13 - Q4: Longest Increasing Subsequence**MEDIUM-HARD · Q4**

In the "longest streak" round, players are given a sequence of numbers and must find the longest strictly increasing streak they can pick out (not necessarily consecutive). Read N integers and print the length of the longest strictly increasing subsequence. Aim for $O(N \log N)$.

Input	Line1: N . Line2: N integers.
Output	Length of the LIS.
Constraints	$1 \leq N \leq 10^5$
Sample Input	6 10 9 2 5 3 7
Sample Output	3 (2, 5, 7 or 2, 3, 7)

Set 13 - Q5: Edit Distance**HARD · Q5**

For the "word transform" round, players must compute the minimum number of edits needed to turn one word into another. Given two strings A and B , print the minimum number of insertions, deletions, or substitutions needed to convert A into B .

Input	Two strings A and B on two lines.
Output	Minimum edit distance.
Constraints	$1 \leq A , B \leq 2000$
Sample Input	horse ros
Sample Output	3

Set 13 - Q6: Maximum Sum Increasing Subsequence with Reconstruction**VERY HARD · Q6**

As a bonus challenge, Dip wants not just the maximum-SUM increasing subsequence, but a way to actually reconstruct and display one such winning subsequence on stage. Read N integers. Find the maximum possible SUM (not length) of a strictly increasing subsequence, and also print one valid subsequence achieving it.

Input	Line1: N . Line2: N integers.
Output	Line1: maximum sum. Line2: one valid subsequence achieving it, space-separated.
Constraints	$1 \leq N \leq 10^4$
Sample Input	5 1 101 2 3 100
Sample Output	106 1 2 3 100

Set 13 - Q7: Optimal Binary Search Tree Cost**EXTREME - Q7**

For the grand finale, Dip wants to build an optimal decision tree over frequently-asked puzzle categories, minimizing the average number of questions asked to find any category. Given N keys with access frequencies, build a binary search tree over them (keys must stay in sorted order as a BST) to MINIMIZE the total weighted search cost (sum of frequency $\times$ depth, root at depth 1). Print the minimum cost using interval DP.

Input	Line1: N . Line2: N frequencies (corresponding to keys already given in sorted order).
Output	Minimum weighted search cost.
Constraints	$1 \leq N \leq 500$
Sample Input	3 34 8 50
Sample Output	142

Set 14

Combinatorics & Modular Math

Set 14 - Q1: Factorial Mod P

VERY EASY · Q1

Statistics TA Oindrila is helping students with a combinatorics assignment. First question: compute $N!$ modulo a large prime, since raw factorials overflow instantly. Read N and print $N! \bmod (10^9+7)$.

Input	One integer N .
Output	$N! \bmod (10^9+7)$.
Constraints	$0 \leq N \leq 10^6$
Sample Input	5
Sample Output	120

Set 14 - Q2: $nCr \bmod P$

EASY · Q2

Next: compute " N choose R " modulo the same prime, for use in a binomial-probability example. Read N and R . Print $C(N,R) \bmod (10^9+7)$ using precomputed factorials and modular inverse (Fermat's little theorem), not Pascal's triangle, since N can be large.

Input	Two integers N R .
Output	$C(N,R) \bmod (10^9+7)$.
Constraints	$0 \leq R \leq N \leq 10^6$
Sample Input	5 2
Sample Output	10

Set 14 - Q3: Count Arrangements with Repetition

MEDIUM · Q3

A student asks how many distinct ways letters of a word (with repeats) can be rearranged — Oindrila needs to count that modulo the prime too. Given a string with possibly repeated letters, print the number of DISTINCT permutations of it, mod (10^9+7) .

Input	One string S .
Output	Number of distinct permutations mod (10^9+7) .
Constraints	$1 \leq S \leq 10^6$
Sample Input	aab
Sample Output	3

Set 14 - Q4: Modular Exponentiation

MEDIUM-HARD · Q4

For a cryptography side-example, Oindrila needs fast modular exponentiation: base raised to a huge exponent, modulo a given number. Read base B , exponent E , and modulus M . Print $B^E \bmod M$ using fast exponentiation ($O(\log E)$), not a loop of E multiplications.

Input	Three integers B E M .
Output	$B^E \bmod M$.
Constraints	$0 \leq B \leq 10^9$, $0 \leq E \leq 10^{18}$, $1 \leq M \leq 10^9$
Sample Input	2 10 1000
Sample Output	24

Set 14 - Q5: Number of Ways to Tile a $2 \times N$ Board with 1×2 Dominoes, Modulo P**HARD · Q5**

For a tiling puzzle in the textbook, Oindrila must count the number of ways to fully tile a $2 \times N$ board using only 1×2 dominoes. Given N , print the number of ways to completely tile a $2 \times N$ board using 1×2 (or 2×1) dominoes, modulo 10^9+7 . (This reduces to a Fibonacci-like recurrence — derive it yourself rather than assuming the answer.)

Input	One integer N .
Output	Number of tilings mod (10^9+7) .
Constraints	$1 \leq N \leq 10^6$
Sample Input	3
Sample Output	3

Set 14 - Q6: Catalan Number Application — Valid Parenthesizations**VERY HARD · Q6**

The same textbook chapter asks for the number of valid balanced-parenthesis arrangements of N pairs — a classic Catalan-number application. Given N pairs of parentheses, print the number of distinct valid combinations, modulo 10^9+7 , using the Catalan number formula $C(2N,N)/(N+1)$ computed via modular inverse (not by generating all strings).

Input	One integer N .
Output	Number of valid combinations mod (10^9+7) .
Constraints	$1 \leq N \leq 10^6$
Sample Input	3
Sample Output	5

Set 14 - Q7: Derangements Count (no fixed points), mod P**EXTREME · Q7**

Finally, for the "Secret Santa" example (nobody should draw their own name), Oindrila needs to count derangements of N people, modulo the prime. Given N , print the number of permutations of $1..N$ in which NO element appears in its original position (derangements), modulo 10^9+7 . Derive and use the recurrence $D(n) = (n-1) \cdot (D(n-1) + D(n-2))$ rather than brute-force generating permutations.

Input	One integer N .
Output	$D(N)$ mod (10^9+7) .
Constraints	$0 \leq N \leq 10^6$
Sample Input	4
Sample Output	9

Set 15

Advanced Greedy + Prefix Sums (World T20 X Team style)

Set 15 - Q1: Maximize Sum of Pairs (fixed pairs)

VERY EASY · Q1

Team manager Zayan is picking cricket pairs for a fun exhibition match, and wants to pair up all players to MAXIMIZE the total skill-gap excitement across all pairs. Given $2N$ integers, pair them up into N pairs to MAXIMIZE the sum of $|\text{difference}|$ across all pairs. Print the maximum total. (Hint: sort first.)

Input	Line1: $2N$. Line2: $2N$ integers.
Output	Maximum total sum of pairwise absolute differences.
Constraints	$1 \leq N \leq 5 \times 10^4$
Sample Input	4 1 5 3 8
Sample Output	9

Set 15 - Q2: Maximum Sum of Chosen Subset's Adjacent Differences

EASY · Q2

For a smaller exhibition team of exactly K players (not paired, just a group), Zayan wants to maximize the sum of skill-gaps between every two players in that group. Given N sorted-after-you-sort integers, choose any subset of size K to MAXIMIZE the sum of absolute differences between all $C(K,2)$ pairs in the subset. Print the maximum sum for a given K .

Input	Line1: N K . Line2: N integers.
Output	Maximum sum.
Constraints	$1 \leq K \leq N \leq 10^5$
Sample Input	5 3 1 2 3 4 5
Sample Output	8

Set 15 - Q3: World T20 X Team (full version)

MEDIUM · Q3

This is the real deal: for the actual World T20 tournament, Zayan must answer many queries, each asking for a team of size X that maximizes the total pairwise rating gap, and needs every query answered instantly. Given N player ratings, answer Q queries. Each query gives X (team size). For that X , choose X players to MAXIMIZE the sum of $|\text{rating}_i - \text{rating}_j|$ over all pairs in the chosen team. Print the answer for each query. Precompute using sorted prefix sums so each query answers in $O(1)$ or $O(\log N)$.

Input	Line1: N . Line2: N integers. Line3: Q . Next Q lines: X .
Output	Q lines: maximum sum for each X .
Constraints	$1 \leq N, Q \leq 10^5$
Sample Input	5 1 2 3 4 5 2 2 4
Sample Output	4 10

Set 15 - Q4: Minimum Cost to Connect Sticks (via extremes, priority-queue idea)**MEDIUM-HARD · Q4**

In the equipment room, Zayan must repeatedly combine bamboo practice sticks (cost = sum of the two combined lengths) until only one remains, and wants to MINIMIZE total combining cost. Given N stick lengths, repeatedly combine any two sticks into one (cost = sum of their lengths, and the new stick replaces both) until one stick remains. Minimize total cost. (Think: is picking the two SHORTEST sticks each time optimal? Prove it or find a counterexample before coding.)

Input	Line1: N . Line2: N integers.
Output	Minimum total cost.
Constraints	$1 \leq N \leq 10^5$
Sample Input	4 2 4 3 1
Sample Output	19

Set 15 - Q5: Balanced Partition (Minimize Difference)**HARD · Q5**

For a fair scrimmage, Zayan wants to split the full squad into two teams whose total skill ratings are as close as possible. Given N integers, split them into two groups (not necessarily equal size) to MINIMIZE the absolute difference between the two groups' sums. Print the minimum difference. (N is small enough here for a DP/subset-sum approach — think about why the greedy "largest first" approach does NOT work for this one, unlike the earlier problems in this set.)

Input	Line1: N . Line2: N integers.
Output	Minimum possible difference.
Constraints	$1 \leq N \leq 30$, $1 \leq a_j \leq 1000$
Sample Input	4 1 6 11 5
Sample Output	1

Set 15 - Q6: Task Scheduling with Cooldown (greedy + counting)**VERY HARD · Q6**

Between matches, players need rest between repeat appearances of the same role — Zayan must schedule to minimize the total match slots needed given this cooldown rule. Given N tasks (represented as letters) and a cooldown period K (the same task type must wait at least K units before repeating), print the minimum total time units needed to complete all tasks (idle slots allowed).

Input	Line1: N K . Line2: a string of N task letters.
Output	Minimum total time units.
Constraints	$1 \leq N \leq 10^4$, $0 \leq K \leq 100$
Sample Input	6 2 AAABBB
Sample Output	8

Set 15 - Q7: World T20 X Team, Offline with Point Updates (hardest variant)**EXTREME - Q7**

Just before the tournament, player ratings keep getting updated live (injuries, form changes) right up until each team-selection query — Zayan needs the team-selection queries answered correctly even with ratings updating in between. Same as the full World T20 X Team problem (Set 15 Q3), but now ratings can be UPDATED between queries: process a mixed stream of "U i v" (update player i's rating to v) and "Q X" (query the max pairwise-difference sum for team size X) operations, answering each Q operation online (in the given order). Think carefully about whether a full offline batch precomputation still works when updates are interleaved, or whether you need a different data structure (e.g. a Fenwick tree over sorted rating buckets) to keep queries fast.

Input	Line1: N. Line2: N integers (initial ratings). Line3: Q. Next Q lines: "U i v" or "Q X".
Output	One line per Q operation: the requested maximum sum.
Constraints	$1 \leq N, Q \leq 10^5$
Sample Input	5 1 2 3 4 5 2 Q 2 U 1 100
Sample Output	4

Set 16

Advanced Graphs — Implicit Graphs & Multi-source

Set 16 - Q1: Multi-Source BFS (Nearest Fire)

VERY EASY · Q1

Firefighter dispatcher Lamia is mapping a city grid where fire has broken out at multiple points, and needs the distance from EVERY cell to its nearest fire, for evacuation planning. Given an $N \times M$ grid where some cells are 'F' (fire) and the rest are '.', print for every cell the minimum distance (in grid steps, 4-directional) to the nearest fire cell.

Input	Line1: N M. Next N lines: grid rows.
Output	N lines, M integers each: distance grid.
Constraints	$1 \leq N, M \leq 500$
Sample Input	<pre>3 3 F</pre>
Sample Output	<pre>2 1 2 1 0 1 2 1 2</pre>

Set 16 - Q2: Knight's Minimum Moves

EASY · Q2

A chess enthusiast on Lamia's team wonders: what's the minimum number of knight moves between two arbitrary squares on an infinite board? On an infinite chessboard, given a knight's start $(x1, y1)$ and target $(x2, y2)$, print the minimum number of knight moves to reach the target. Use BFS with pruned bounds around the two points.

Input	Four integers $x1\ y1\ x2\ y2$.
Output	Minimum number of knight moves.
Constraints	$-300 \leq \text{coords} \leq 300$
Sample Input	0 0 3 3
Sample Output	2

Set 16 - Q3: SHA75-style Full Reachability with Precomputation

MEDIUM · Q3

Back to disaster recovery — Lamia now needs to know, for EVERY possible starting machine in the failover network (from Set 12), whether the primary server (node N) is reachable, without re-running the search from scratch each time. Same jump-graph setup as before (from i you can go to $i+A[i]$ or $i-A[i]$), but now you must answer whether EVERY pair (start, N) is reachable, for $\text{start} = 1..N$, efficiently — precompute reachability to node N for all starting nodes using a SINGLE reverse BFS from node N (think about reversing the edges), rather than running BFS N times.

Input	Line1: N. Line2: N integers (array A).
Output	N characters (or space-separated 1/0): whether node i can reach node N.
Constraints	$1 \leq N \leq 10^5$
Sample Input	<pre>5 2 3 1 1 1</pre>
Sample Output	1 0 1 1 1 (example — verify by hand)

Set 16 - Q4: 0-1 BFS (Grid with Some Free Moves)

MEDIUM-HARD · Q4

For the city's evacuation grid, some intersections are marked as free "shuttle points" (zero-cost to move through) — Lamia needs the minimum-cost evacuation path considering these free shortcuts. Given an $N \times M$ grid, moving to an adjacent cell normally costs 1, but moving onto a cell marked 'T' (teleport-friendly) costs 0. Find the minimum cost path from top-left to bottom-right. Use a deque-based 0-1 BFS, not Dijkstra with a heap (unnecessary overhead here).

Input	Line1: N M. Next N lines: grid rows ('.', '#', 'wall', 'T').
Output	Minimum cost, or -1 if unreachable.
Constraints	$1 \leq N, M \leq 500$
Sample Input	<pre>2 2 . T # .</pre>
Sample Output	0

Set 16 - Q5: Shortest Cycle Through a Given Edge / Node (mini)

HARD · Q5

Lamia also wants to find the shortest possible loop route that a patrol car can take that necessarily passes back through a specific checkpoint intersection. Given an undirected graph with N nodes and M edges, and a specific node K , print the length of the SHORTEST cycle that passes through node K , or -1 if no such cycle exists. (Hint: try BFS from each neighbor of K , or think about removing K 's edges one at a time and running BFS between its neighbors.)

Input	Line1: N M K. Next M lines: u v.
Output	Shortest cycle length through K, or -1.
Constraints	$1 \leq N \leq 500, 0 \leq M \leq 2000$
Sample Input	<pre>4 4 1 1 2 2 3 3 4 4 1</pre>
Sample Output	4

Set 16 - Q6: Dijkstra with Path Reconstruction on Large Sparse Graph

VERY HARD · Q6

For emergency supply routing, roads have different travel costs, and Lamia needs both the shortest cost AND the actual route from the depot to the affected zone. Given a weighted directed graph with N nodes and M edges (non-negative weights), a source S , and a target T , print the shortest distance from S to T AND one valid shortest path (sequence of nodes). Use a priority-queue-based Dijkstra for $O((N+M) \log N)$; N and M can be up to 2×10^5 .

Input	Line1: N M S T. Next M lines: u v w.
Output	Line1: shortest distance, or -1 if unreachable. Line2 (if reachable): the path, space-separated node ids.
Constraints	$1 \leq N \leq 2 \times 10^5, 0 \leq M \leq 2 \times 10^5, 0 \leq w \leq 10^4$
Sample Input	<pre>4 4 1 4 1 2 1 2 3 2 3 4 1 1 4 10</pre>
Sample Output	<pre>4 1 2 3 4</pre>

Set 16 - Q7: Reachability Precomputation on Jump Graph With Weighted Cost (final boss graph problem)
EXTREME - Q7

Finally, combining ideas: in the failover network, some large "safe-hub" machines allow free jumps, and Lamia needs the minimum total cost to reach a backup machine from any starting point, given this mix of free and costly jumps. Extend the SHA75-style jump graph: from index i you may move to $i+A[i]$ or $i-A[i]$, but EACH move costs 1 unit, UNLESS you move to an index that is a multiple of a given value M , in which case that specific move costs 0. Answer Q queries of (start, end): print the minimum cost to go from start to end, or -1 if unreachable. Use 0-1 BFS (deque-based) since edge weights are only 0 or 1.

Input	Line1: N M . Line2: N integers (array A). Line3: Q . Next Q lines: start end.
Output	Q lines: minimum cost, or -1.
Constraints	$1 \leq N \leq 10^5$, $1 \leq Q \leq 1000$
Sample Input	<pre> 5 2 2 3 1 1 1 1 1 5 </pre>
Sample Output	<pre> 1 </pre>

Set 17

Advanced Number Theory

Set 17 - Q1: Sum of Divisors Function

VERY EASY · Q1

Cryptography club member Rafid is exploring number properties. First exercise: compute the sum of all divisors of a given number. Read N . Print the sum of all divisors of N (including 1 and N itself), computed in $O(\sqrt{N})$.

Input	One integer N .
Output	Sum of divisors of N .
Constraints	$1 \leq N \leq 10^{12}$
Sample Input	28
Sample Output	56

Set 17 - Q2: Euler's Totient Function

EASY · Q2

Next, Rafid needs Euler's totient function — counting integers up to N that share no common factor with N . Read N . Print $\phi(N)$, the count of integers in $[1, N]$ coprime with N , computed via prime factorization in $O(\sqrt{N})$.

Input	One integer N .
Output	$\phi(N)$.
Constraints	$1 \leq N \leq 10^{12}$
Sample Input	9
Sample Output	6

Set 17 - Q3: Modular Inverse

MEDIUM · Q3

For an RSA-style encryption demo, Rafid needs the modular inverse of a number under a prime modulus. Read A and M (M is prime). Print the modular inverse of A modulo M , i.e., a value X such that $(A \cdot X) \bmod M = 1$.

Input	Two integers A M .
Output	The modular inverse.
Constraints	$1 \leq A < M \leq 10^9$, M prime
Sample Input	3 11
Sample Output	4

Set 17 - Q4: Count Pairs with a Given GCD**MEDIUM-HARD · Q4**

Rafid's club is analyzing a dataset and wants to count exactly how many PAIRS of numbers in it have a GCD equal to a specific target value. Given N and an integer array of size N , and an integer G , count the number of pairs (i, j) , $i < j$, such that $\gcd(a_i, a_j) = G$ exactly. Use a divisor-based counting technique (count multiples of each d , apply inclusion-exclusion / Möbius-style subtraction from large d to small d) rather than checking all $O(N^2)$ pairs.

Input	Line1: N G . Line2: N integers.
Output	Count of pairs with gcd exactly G .
Constraints	$1 \leq N \leq 10^5$, $1 \leq a_i \leq 10^5$
Sample Input	4 2 2 4 6 3
Sample Output	2

Set 17 - Q5: Maximum GCD Subsequence of Length K (full version)**HARD · Q5**

Revisiting the earlier subsequence puzzle at full scale, Rafid now needs FAST answers (across many queries) for the maximum GCD of any increasing subsequence of a given length. Read N integers and Q queries, each giving K . For each query, print the maximum GCD achievable over any increasing subsequence of length exactly K (or -1 if impossible). This is the FULL version of Set 10 Q5 and mirrors the hardest problem ("GCD") from your real contest — for each divisor d from high to low, run LIS DP restricted to multiples of d , and combine answers for all K in one pass.

Input	Line1: N . Line2: N integers. Line3: Q . Next Q lines: K .
Output	Q lines: max GCD for that K , or -1.
Constraints	$1 \leq N, Q \leq 3000$, $1 \leq a_i \leq 3000$
Sample Input	5 4 8 6 12 3 2 2 4
Sample Output	4 -1

Set 17 - Q6: Sum of GCDs of All Subarrays**VERY HARD · Q6**

For a research project, Rafid needs the sum of GCD values across every possible contiguous subarray of a large dataset. Given N integers, compute the sum of $\gcd(a[l..r])$ over ALL $O(N^2)$ subarrays, in $O(N \log(\max A))$ using the fact that GCD values along a fixed left endpoint change $O(\log(\max A))$ times as the right endpoint grows.

Input	Line1: N . Line2: N integers.
Output	Sum of GCD over all subarrays.
Constraints	$1 \leq N \leq 10^5$, $1 \leq a_i \leq 10^9$
Sample Input	3 2 4 6
Sample Output	$2+2+2+4+2+6=18$ (verify by listing subarrays)

Set 17 - Q7: Smallest Multiple With Only Digits 0 and 1 (BFS on remainders)**EXTREME · Q7**

Finally, Rafid is curious: what's the smallest multiple of a number N whose digits are only 0s and 1s? He wants an efficient way to find it, not brute-force checking multiples one by one. Given N , find the smallest positive multiple of N that consists only of the digits 0 and 1 (e.g. for $N=3$, answer is 111). Model this as a shortest-path/BFS problem over the N possible remainders mod N , since brute-force checking multiples directly is infeasible for large N .

Input	One integer N .
Output	The smallest such multiple, as a string (may be very long).
Constraints	$1 \leq N \leq 10^5$
Sample Input	3
Sample Output	111

Set 18

Advanced Dynamic Programming

Set 18 - Q1: Coin Change — Minimum Coins

VERY EASY · Q1

Vending-machine engineer Sabbir needs the minimum number of coins to dispense any given change amount, from a fixed set of coin types. Given N coin denominations and a target amount X , print the minimum number of coins needed to make exactly X , or -1 if impossible.

Input	Line1: N X . Line2: N denominations.
Output	Minimum coins, or -1 .
Constraints	$1 \leq N \leq 100$, $1 \leq X \leq 10^4$
Sample Input	3 11 1 2 5
Sample Output	3

Set 18 - Q2: Longest Common Subsequence

EASY · Q2

Sabbir is also comparing two barcodes (as strings) and needs their longest common subsequence for a fuzzy-matching feature. Given two strings A and B , print the length of their longest common subsequence.

Input	Two strings A and B .
Output	Length of LCS.
Constraints	$1 \leq A , B \leq 2000$
Sample Input	abcde ace
Sample Output	3

Set 18 - Q3: Matrix Chain Multiplication (min scalar multiplications)

MEDIUM · Q3

For a factory-automation problem, Sabbir must decide the cheapest way to multiply a chain of matrices representing sequential transformations. Given the dimensions of N matrices to multiply in order, print the minimum number of scalar multiplications needed to compute the product, using optimal parenthesization.

Input	Line1: N . Line2: $N+1$ integers (dimensions $p_0..p_N$).
Output	Minimum scalar multiplications.
Constraints	$1 \leq N \leq 100$
Sample Input	3 10 20 30 40
Sample Output	18000

Set 18 - Q4: Longest Palindromic Subsequence**MEDIUM-HARD · Q4**

Sabbir's team is compressing sensor-log strings and wants the longest palindromic PATTERN (subsequence, not necessarily contiguous) hidden in a log. Given a string S , print the length of its longest palindromic subsequence.

Input	One string S .
Output	Length of the longest palindromic subsequence.
Constraints	$1 \leq S \leq 2000$
Sample Input	bbbab
Sample Output	4

Set 18 - Q5: Maximum GCD-LIS Combined (final boss, matches Problem J exactly)**HARD · Q5**

This is the big one — Sabbir must combine everything: for a stream of sensor values, find for every possible subsequence length the maximum GCD achievable, precomputed efficiently for all lengths at once. Read N integers and Q queries, each giving K ($1 \leq K \leq N$). For each query print the maximum GCD achievable over any strictly increasing subsequence of length exactly K , or -1 if none exists. N and Q can be up to 10^5 , and a_i up to 10^5 — you must precompute answers for ALL K from 1 to N in roughly $O((N + \max A) \log(\max A))$ or similar, not per-query $O(N^2)$. This is a direct rebuild of the hardest problem from your original contest.

Input	Line1: N . Line2: N integers. Line3: Q . Next Q lines: K .
Output	Q lines: answer for each K .
Constraints	$1 \leq N, Q \leq 10^5, 1 \leq a_i \leq 10^5$
Sample Input	6 4 8 6 12 3 16 3 2 3 5
Sample Output	4 4 -1

Set 18 - Q6: Longest Increasing Subsequence With At Most One Deletion**VERY HARD · Q6**

Sabbir also wonders: if he's allowed to discard exactly one faulty sensor reading first, how much longer can the increasing calibration sequence become? Given N integers, find the length of the longest strictly increasing subsequence you can form after deleting AT MOST ONE element from the array first (deletion is from the original array, then take an increasing subsequence of what remains).

Input	Line1: N . Line2: N integers.
Output	Length of the best achievable increasing subsequence.
Constraints	$1 \leq N \leq 10^5$
Sample Input	6 5 1 2 3 4 9
Sample Output	5

Set 18 - Q7: Maximum GCD-Sum Subsequence (weighted variant)**EXTREME - Q7**

Finally, factoring in reliability WEIGHTS per sensor, Sabbir wants the increasing subsequence that maximizes (GCD of readings) $\times$ (total reliability weight). Read N integers, each also with a weight w_i . Among all strictly increasing subsequences, find the one that MAXIMIZES (GCD of chosen elements) $\times$ (sum of their weights). Print this maximum value. This combines the divisor-DP idea with an additional DP dimension for weight — think carefully about state design before coding.

Input	Line1: N . Line2: N integers (values). Line3: N integers (weights).
Output	The maximum achievable (GCD $\times$ weight-sum).
Constraints	$1 \leq N \leq 2000$, $1 \leq a_i, w_i \leq 2000$
Sample Input	4 2 4 8 3 1 1 1 1
Sample Output	2*3=6 (subsequence 2,4,8 has GCD 2 and weight-sum 3)

Set 19

Mixed Practice Contest — Round A

Set 19 - Q1: Ticket Number (warm-up)

VERY EASY · Q1

Bus conductor Antu collects ticket numbers all day and has heard that "lucky" tickets (where digit-sums of both halves match) bring good fortune — help him check any ticket instantly. A ticket number is "lucky" if the sum of its first half digits equals the sum of its second half digits. Given a ticket number with an even number of digits, print "LUCKY" or "NOT LUCKY".

Input	One string of digits, even length.
Output	"LUCKY" or "NOT LUCKY".
Constraints	$2 \leq \text{length} \leq 20$, even
Sample Input	123402
Sample Output	LUCKY

Set 19 - Q2: Minimum Deletions to Make Frequency Unique

EASY · Q2

For inventory purposes, Antu wants to trim a list of item codes (a string) so that no two different codes share the exact same usage frequency. Given a string, print the minimum number of characters to delete so that no two distinct characters have the same frequency.

Input	One string S of lowercase letters.
Output	Minimum deletions.
Constraints	$1 \leq S \leq 10^5$
Sample Input	aab
Sample Output	0

Set 19 - Q3: Smallest Window Containing All Characters of Pattern

MEDIUM · Q3

Antu is searching a long log string for the SMALLEST window that contains every character of a given complaint keyword pattern. Given string S and pattern P, find the length of the smallest substring of S that contains all characters of P (with at least the same multiplicity), or -1 if impossible.

Input	Two strings S and P.
Output	Minimum window length, or -1.
Constraints	$1 \leq P \leq S \leq 10^5$
Sample Input	ADOBECODEBANC ABC
Sample Output	4 ("BANC")

Set 19 - Q4: Maximum Product Subarray**MEDIUM-HARD · Q4**

For a bonus scheme, Antu wants the maximum PRODUCT of ticket sales over any contiguous stretch of days, even though some days had negative (refunded) totals. Given N integers (can be negative or zero), print the maximum product of a contiguous subarray.

Input	Line1: N. Line2: N integers.
Output	Maximum product.
Constraints	$1 \leq N \leq 10^5$, $ a_i \leq 20$
Sample Input	5 2 3 -2 4 -1
Sample Output	48 (product of the full array: $2 \cdot 3 \cdot -2 \cdot 4 \cdot -1$)

Set 19 - Q5: K Closest Points to Origin, then Path Feasibility**HARD · Q5**

For a promotional stunt, Antu picks the K bus stops closest to the depot (by straight-line distance) and wants to know if all K stops can be visited via short connecting hops (distance $\leq D$) without leaving any stop stranded. Given N 2D points and integer K, first find the K points closest to the origin (Euclidean distance). Then, treating those K points as nodes fully connected to each other (edge exists if Euclidean distance $\leq D$, a given threshold), print "YES" if all K points form a single connected component, else "NO".

Input	Line1: N K D. Next N lines: x y.
Output	"YES" or "NO".
Constraints	$1 \leq K \leq N \leq 2000$
Sample Input	4 3 5 0 0 3 4 1 1 100 100
Sample Output	YES

Set 19 - Q6: Sliding Window Maximum, With Sum-of-Maximums Twist**VERY HARD · Q6**

Antu is analyzing peak passenger counts across sliding time windows on his route and wants the SUM of the maximum passenger count in every window of a fixed size. Given N integers and window size K, for every contiguous window of size K compute its maximum, and print the SUM of all these window-maximums. Use a monotonic deque for $O(N)$, not a heap or $O(N \cdot K)$ brute force.

Input	Line1: N K. Line2: N integers.
Output	Sum of all window maximums.
Constraints	$1 \leq K \leq N \leq 10^5$
Sample Input	8 3 1 3 -1 -3 5 3 6 7
Sample Output	36 (window maxes: 3, 3, 5, 5, 6, 7)

Set 19 - Q7: Trapping Rain Water in 2D (advanced grid + priority queue)**EXTREME - Q7**

Finally, after a heavy storm, Antu's team is mapping how much rainwater pools across the city's uneven terrain grid, and needs the total trapped volume computed. Given an $N \times M$ grid of non-negative heights representing terrain, compute the total volume of rainwater trapped after rain (water is trapped in a cell if it's surrounded by higher or equal terrain in all directions, following boundary constraints). Use a min-heap based boundary expansion (similar to Dijkstra), $O(NM \log(NM))$.

Input	Line1: N M. Next N lines: M integers each (heights).
Output	Total trapped water volume.
Constraints	$1 \leq N, M \leq 200$
Sample Input	<pre>3 6 1 4 3 1 3 2 3 2 1 3 2 4 2 3 3 2 3 1</pre>
Sample Output	<pre>4</pre>

Set 20

Final Mock Contest — Round B (Cumulative, hardest set)

Set 20 - Q1: Array Product Without Division

VERY EASY · Q1

Head judge Wasif is finalizing the university's flagship contest problem set and starts with a classic: given N numbers, report for each position the product of all OTHER numbers, without ever dividing. Given N integers, print an array where $\text{output}[i] = \text{product of all elements except } a[i]$, WITHOUT using division, in $O(N)$ time.

Input	Line1: N . Line2: N integers.
Output	N integers, space-separated.
Constraints	$1 \leq N \leq 10^5$, values fit in signed 64-bit after multiplication
Sample Input	4 1 2 3 4
Sample Output	24 12 8 6

Set 20 - Q2: Median in a Stream

EASY · Q2

As scores stream in live during the contest, Wasif's live scoreboard must report the running MEDIAN score after each new submission arrives. Process N integers one at a time (arriving in the given order). After each insertion, print the current median of all numbers seen so far (if even count, print the average of the two middle values, as a value with up to 1 decimal place if needed). Use two heaps (or an ordered structure) for $O(\log N)$ per insertion, not a full re-sort each time.

Input	Line1: N . Line2: N integers (the stream, in order).
Output	N lines: median after each insertion.
Constraints	$1 \leq N \leq 10^5$
Sample Input	3 5 15 1
Sample Output	5 10 5

Set 20 - Q3: Maximum GCD Subsequence + Range Team Query, Combined

MEDIUM · Q3

For the marquee hardest problem, Wasif combines two classic ideas into one: querying both the max GCD of a length- K increasing subsequence, and the max pairwise-difference sum for a size- X team, all in a single problem. Given N integers, answer Q queries of two types: type 1 "K" — print the maximum GCD over any increasing subsequence of length K (as in Set 18 Q5); type 2 "X" — print the maximum sum of pairwise |differences| for a chosen team of size X (as in Set 15 Q3). This directly recreates having to solve BOTH of your original contest's hardest problems (H and J) under one time budget.

Input	Line1: N . Line2: N integers. Line3: Q . Next Q lines: "1 K" or "2 X".
Output	Q lines: the requested answer per query.
Constraints	$1 \leq N, Q \leq 10^5$, $1 \leq a_i \leq 10^5$
Sample Input	5 4 8 6 12 3 2 1 3 2 2
Sample Output	4 9

Set 20 - Q4: Shortest Path with Exactly K Edges

MEDIUM-HARD · Q4

For the closing ceremony's trivia round, Wasif needs the shortest-cost route between two nodes using EXACTLY K hops on the university network graph — no more, no fewer. Given a weighted directed graph with N nodes and M edges, and a source S, target T, and integer K, print the minimum total weight of a path from S to T that uses EXACTLY K edges (edges/nodes may repeat), or -1 if impossible.

Input	Line1: N M K S T. Next M lines: u v w (directed edge $u \rightarrow v$, weight w).
Output	Minimum weight path using exactly K edges, or -1.
Constraints	$1 \leq N \leq 100$, $K \leq 1000$
Sample Input	4 4 2 1 4 1 2 1 2 3 1 3 4 1 1 4 100
Sample Output	2

Set 20 - Q5: Segment Tree with Range Update + Range GCD Query

HARD · Q5

For scoreboard analytics, Wasif's system must support both updating individual scores AND querying the GCD across ranges of scores, fast, as the contest is still running. Read N integers and Q operations: "U l r v" (add v to every element in [l,r]) or "Q l r" (print the GCD of all elements in [l,r]). Implement a segment tree with lazy propagation supporting both range updates and range GCD queries in $O(\log N)$ each — this needs a custom merge (GCD of children) combined with careful lazy-add propagation (note: GCD doesn't distribute over addition the way sum does, so think about how lazy values interact with the stored GCDs).

Input	Line1: N Q. Line2: N integers. Next Q lines: "U l r v" or "Q l r".
Output	One line per Q operation: the requested GCD.
Constraints	$1 \leq N, Q \leq 10^5$
Sample Input	5 2 4 8 12 16 20 Q 1 3 Q 2 4
Sample Output	4 4

Set 20 - Q6: Final Boss: Team Selection Under GCD Constraint (H + J truly combined)

VERY HARD · Q6

As the true final boss of the problem set, Wasif fuses the two hardest ideas from the actual contest into one: choose exactly K players whose rating GCD is divisible by G, and among those, maximize the pairwise rating-difference sum. Read N player ratings. Answer Q queries, each giving (K, G): among all subsequences of EXACTLY K players (not necessarily increasing this time — any subsequence by original order/index) whose GCD of ratings is a multiple of G, find the maximum possible sum of pairwise |rating differences|. Print -1 if no valid subsequence of size K with GCD divisible by G exists. This genuinely fuses the two hardest ideas from your real contest (H's extremal pairwise-difference technique and J's divisor-based subsequence selection) into one problem — expect this to take real design effort, not just recall of a known template.

Input	Line1: N. Line2: N integers. Line3: Q. Next Q lines: K G.
Output	Q lines: the maximum sum, or -1.
Constraints	$1 \leq N$, $Q \leq 2000$, $1 \leq a_i \leq 2000$
Sample Input	5 4 8 6 12 3 2 3 2 2 5
Sample Output	24 -1

Set 20 - Q7: Full Contest Simulator: Everything Combined

EXTREME · Q7

For the very last slot, Wasif leaves it open: design and fully write your own original problem statement combining at least three ideas from this entire practice sheet, in true contest-author style, and then solve it yourself within 45 minutes. Design and solve a single problem of your own that combines at least THREE ideas from this practice sheet (e.g., binary search on the answer + prefix sums + number theory, or graph BFS + greedy + DP). Write the full problem statement (context, input, output, constraints, sample) yourself, then solve it in C within 45 minutes. This exercise trains the hardest and most important contest skill: reading/creating a problem's structure under time pressure, not just coding a known pattern.

Note: This is a self-authored problem — the exam-style discipline of statement comprehension is itself part of the training.

